Amazon SDE II System Design

Deep Dive: E-commerce Order Placement and Inventory

An elaborated study guide for Question 3 of the Top 5 guide: plain-language explanations, ten diagrams, worked examples, a High-Level Design (HLD), a Low-Level Design (LLD) with tested Java, and a section that links the patterns to a supply chain team.

What is inside

Part	You will be able to
A and B: Problem and numbers	State the three promises (no oversell, no double charge, no lost order), size the system, and choose consistency per service
C: HLD	Draw the architecture, run the checkout saga on a whiteboard, explain compensation and the order state machine
D: LLD	Explain a small, tested class design for orders, inventory reservation and idempotent checkout
E: Deep dives	Handle the oversell race, hot items, idempotency, the outbox, payment timeouts and reservation expiry
F, G, H: Example, supply chain, rehearsal	Walk one order end to end, map the patterns to supply chain problems, and answer follow-up questions

The 45-minute shape

Minutes	Step	What to produce
0 to 5	Clarify	Scope, assumptions, what matters most (correctness of stock and money)
5 to 10	Estimate	Orders per second, storage, hot SKU rate, with arithmetic visible
10 to 15	API and consistency map	Endpoints with Idempotency-Key; which service is strong and which is eventual
15 to 28	HLD	Architecture, then the saga, then compensation
28 to 40	Deep dives or LLD	Oversell race, idempotency, outbox; or classes and code if asked
40 to 45	Wrap-up	Failures, reconciliation, monitoring, 10x scaling

> **Amazon-specific tip.** Tie decisions to the customer ("a double charge or a cancelled order is the worst customer experience, so I spend my complexity budget on correctness at checkout and keep the cart simple and always available"), name the bottleneck before the interviewer does, and mention operations: stuck-order alarms and a reconciliation job.

Question 3: Design Order Placement and Inventory for an E-commerce Site

> **How to use this chapter.** Part A explains the problem in plain language. Part B covers requirements and numbers. Part C is the High-Level Design (HLD): the architecture, the checkout saga and the order lifecycle. Part D is the Low-Level Design (LLD): classes and tested Java for orders, inventory reservation and idempotent checkout. Part E goes deep on the hard problems with diagrams. Part F is a full worked example, Part G connects it to a supply chain team, and Part H is rehearsal material.

Part A. Understand the problem first

A customer puts items in a cart, clicks "Place order", pays, and later receives the goods. Behind that click the system must check that the items exist, hold them for this customer, charge the customer, record the order and tell the warehouse to ship. Several independent services are involved, and any of them can be slow, down or retried.

> **Key idea.** Three promises must never be broken: **never sell the same unit twice** (no overselling), **never charge a customer twice** (exactly-once payment effect), and **never lose an order** (durability). Everything in this design exists to keep those three promises while the system is distributed, busy and partly failing.

Why this is harder than it looks

Hard truth	What goes wrong if ignored
Two buyers want the last unit at the same instant	Both are told "success". One order must be cancelled later and the customer loses trust.
Networks time out and clients retry	The same "place order" request arrives twice. Without protection the customer is charged twice.
One database transaction cannot span Inventory, Payment and Orders	A crash in the middle leaves stock reserved with no order, or a payment with no order.
Events can be lost or duplicated	The warehouse is never told to ship, or ships twice.
A flash sale sends thousands of buyers to one item	One database row becomes a bottleneck and everything slows down.

Vocabulary

Term	Meaning
SKU	Stock keeping unit: one distinct product variant (for example "blue T-shirt, size M").
Fulfillment center (FC)	A warehouse that stores stock and ships orders.
Reservation (hold)	Stock set aside for an order that is being paid. Not sold yet, no longer available to others.
Saga	A multi-step business transaction made of local steps; if a step fails, earlier steps are undone by compensating actions.
Idempotency	Doing the same request twice has the same effect as doing it once.
Outbox	A table in the same database as the business data that records events to publish, so the data and the event are saved atomically.

Part B. Requirements and numbers

B1. Clarifying questions

Question to ask	Why it matters	Assumption to state
One warehouse or many?	Decides how inventory is keyed and how an order is allocated	Many fulfillment centers
Is backorder allowed?	Decides whether "out of stock" is a hard failure	No backorders
Is payment in our system or external?	Decides how we handle timeouts and retries	External payment gateway
Guest checkout? Multiple sellers?	Affects cart identity and settlement	Guests allowed, single seller (out of scope: marketplace)
Scale and peaks?	Drives partitioning and caching	10 million orders per day, 10x peak during sales events
What matters most?	Sets the consistency choices	Correctness of stock and money over raw speed at checkout; cart must always be available

> **Say it like this:** "I will design the cart, checkout, inventory and order services for a many-warehouse retailer with an external payment gateway. The cart favors availability. Checkout favors correctness: no oversell, no double charge, no lost order. I am treating search, recommendations and shipping optimization as out of scope unless you want them."

B2. Back-of-the-envelope estimates, step by step

Quantity	Working	Result
Orders per second (average)	10,000,000 / 86,400 seconds	about 115 orders per second
Orders per second (peak)	10x average	about 1,200 orders per second
Product page views	Assume 100 views per order: 1 billion per day / 86,400	about 11,600 per second, 100,000+ at peak (served by cache and CDN)
Inventory writes at peak	1,200 orders/s x about 3 lines per order (reserve) plus commit or release	roughly 3,600 to 7,200 small writes per second in total (easy if spread over many SKUs)
Order storage	10 million x 2 KB = 20 GB per day; x 365	about 7 TB per year
Inventory rows	Assume 500 million active (SKU, FC) pairs x 100 bytes	about 50 GB (tiny rows, large count)
Single hot SKU in a flash sale	Assume 50,000 buyers within 10 seconds on one SKU	about 5,000 writes per second on one key, which is more than one partition handles (roughly 1,000 writes per second)

> **What the numbers tell you.** Overall volume is modest. The challenges are **correctness** (stock and money) and **skew** (one hot SKU), not total throughput. Saying this out loud shows you used the math to find the real problem.

B3. Each service gets the consistency it needs

A strong answer does not apply one consistency level everywhere. Match each component to the cost of being wrong:

Service	Responsibility	Store and consistency	Why
Cart	Items a user intends to buy	DynamoDB keyed by userId or guest id; **available**, merge on conflict	A cart that cannot be edited loses sales; a briefly stale cart costs nothing
Catalog and pricing	Product details, price	Read-heavy; cache and CDN; **eventually consistent**; re-check price at checkout	Stale descriptions are harmless; stale price is checked again later
Inventory	Available and reserved stock per SKU per FC	**Strongly consistent** conditional updates	Wrong stock means oversell or lost sales
Order	Order records and state machine	Durable store keyed by orderId, index by customer and time; **strong**	Orders must never be lost or corrupted
Payment	Authorize and capture via a gateway	Idempotent calls with our reference; **strong**	Money
Fulfillment, notifications, search index	Downstream reactions	Event-driven; **eventual**	Can lag by seconds without harm

B4. API design with examples

```
PUT    /v1/cart/items/{sku}        body: { "quantity": 2 }
GET    /v1/cart

POST   /v1/orders                  header: Idempotency-Key: 6f1c-9a...
       body: { "cartId": "c-77", "addressId": "a-3", "paymentMethodId": "pm-9" }
       201 Created   { "orderId": "O-123", "status": "CONFIRMED" }
       409 Conflict  { "error": "OUT_OF_STOCK", "sku": "SKU-42" }
       402           { "error": "PAYMENT_DECLINED" }

GET    /v1/orders/{orderId}        -> status: PENDING | RESERVED | CONFIRMED | ...
POST   /v1/orders/{orderId}/cancel
```

The **Idempotency-Key** header is the most important detail. The client generates it once per "place order" intent. If the network times out and the client retries, the server recognizes the key and returns the original result instead of creating a second order or charging again.

Part C. High-Level Design (HLD)

Draw the services as separate boxes with their own stores, then show the Order service running the checkout saga and publishing events through an event bus. Mark which parts are strongly consistent and which are eventual; interviewers notice that distinction.

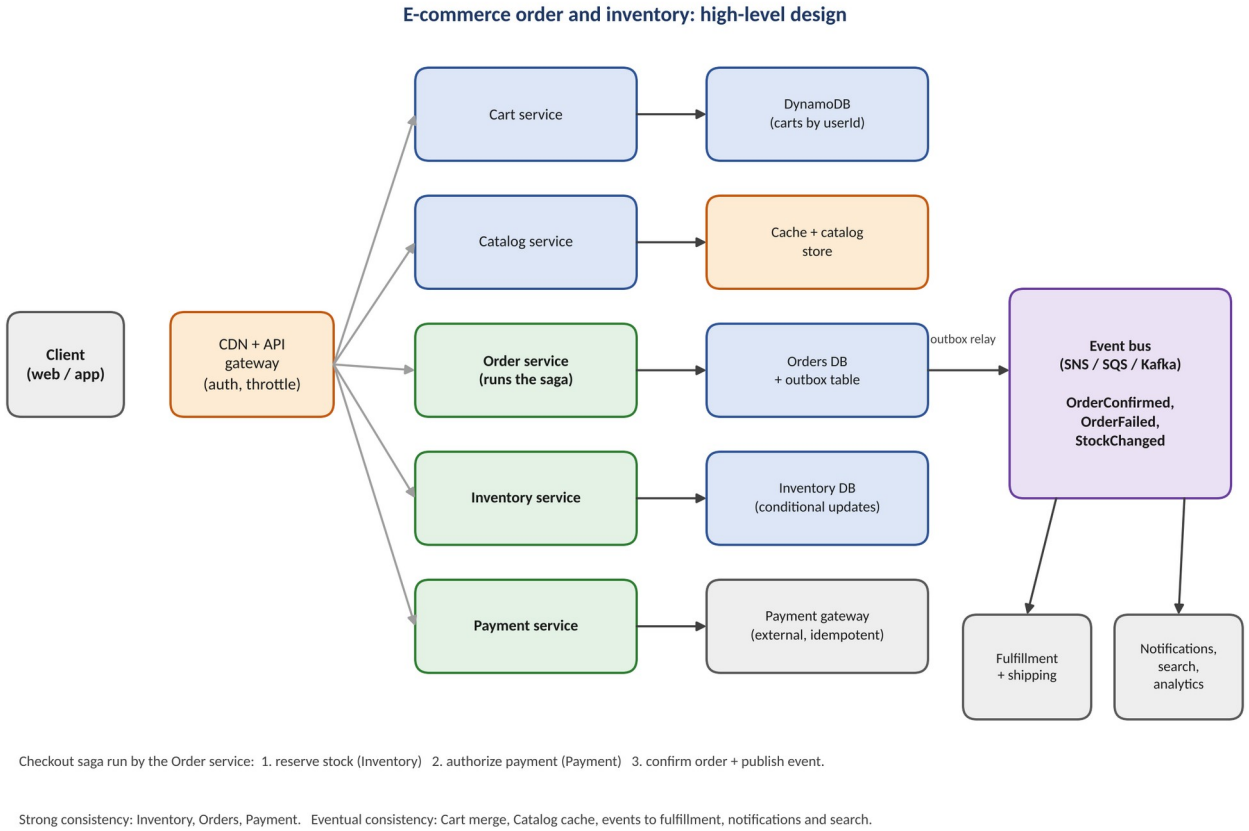


Figure 1. E-commerce HLD: services, their stores and the event bus

C1. Components and why each exists

Component	Responsibility	Technology example	Why it is separate
CDN and API gateway	TLS, authentication, throttling, routing	CloudFront, API Gateway	Absorbs browse traffic; protects services
Cart service	Add, remove, merge items	DynamoDB (key = userId)	Must stay available even if checkout is struggling
Catalog service	Product and price data	Cache plus a catalog store	Read-heavy and cache-friendly
Order service	Order records, state machine, runs the checkout saga	Relational or DynamoDB, plus an outbox table	Owns the order lifecycle and the idempotency keys
Inventory service	Available and reserved counts, reservations	DynamoDB conditional updates or SQL	The correctness-critical, contention-prone component
Payment service	Talks to the external gateway	Wrapper with retries and idempotency	Isolates a slow, unreliable dependency
Event bus	Delivers OrderConfirmed and similar events	SNS + SQS or Kafka	Decouples fulfillment, notifications and search from checkout latency

C2. The checkout saga, happy path

A single database transaction cannot span three services, so checkout is a **saga**: a sequence of local steps, each saved, with a compensating action for each step if a later step fails. The Order service is the orchestrator.

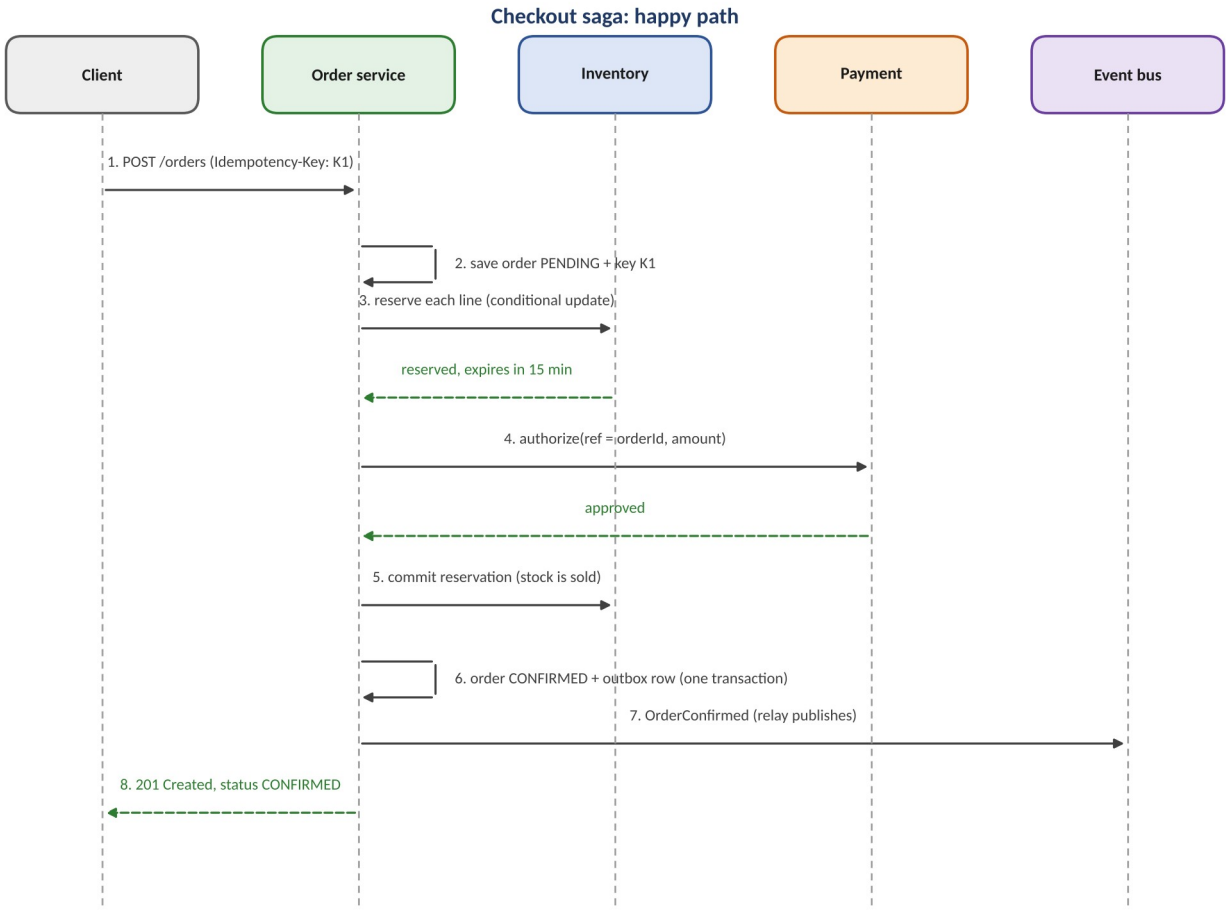


Figure 2. Checkout saga: happy path

1. **Receive and record.** Save the order as PENDING together with the Idempotency-Key. If the key already exists, return the stored result and stop.
2. **Reserve stock.** For every line, atomically move quantity from available to reserved, with an expiry (for example 15 minutes). If any line cannot be reserved, release the lines already reserved and fail the order (all or nothing).
3. **Authorize payment** with the gateway using the order id as the reference, so the gateway can also de-duplicate.
4. **Commit.** Convert the reservation to a sale (reserved goes down, nothing returns to available), mark the order CONFIRMED and write an `OrderConfirmed` row to the outbox in the same transaction.
5. **Publish.** A relay sends the outbox events to the bus; fulfillment, notifications and the search index react.

C3. When something fails: compensation

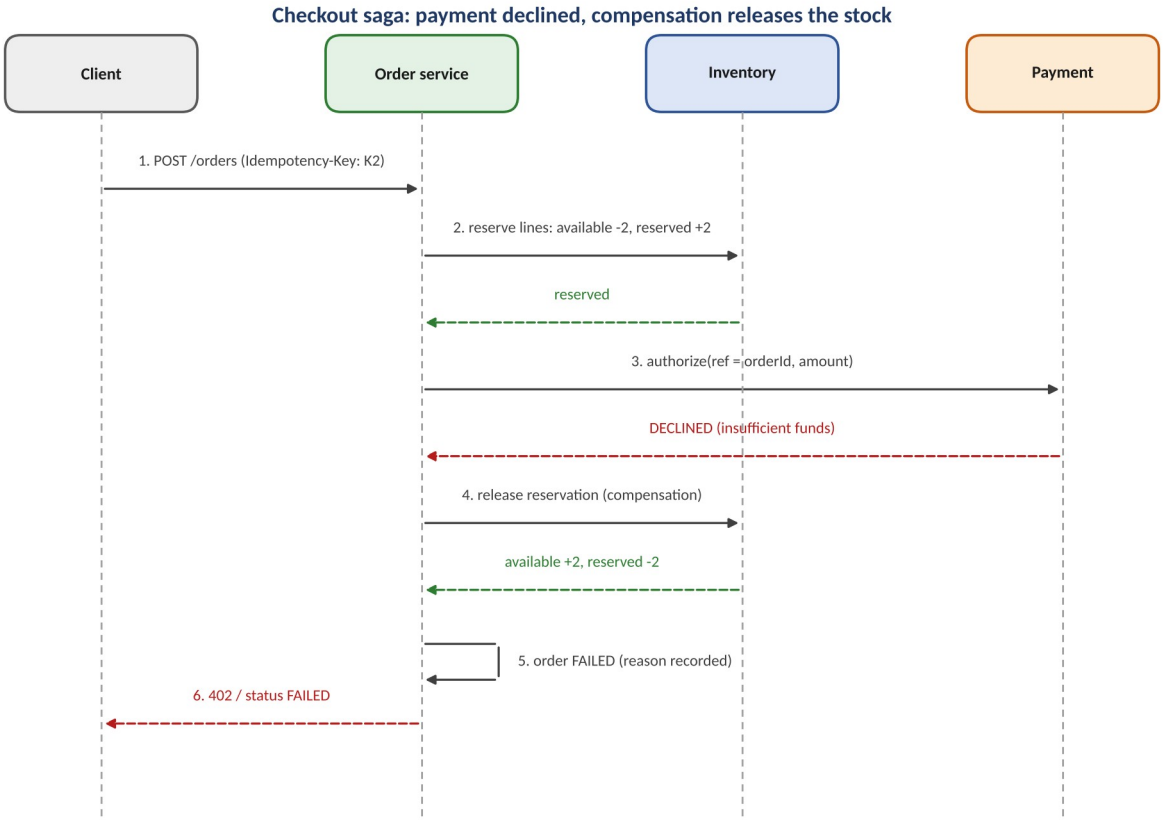


Figure 3. Payment declined: the reservation is released

The rule: every forward step that changes state has an undo. Reserve is undone by release. If the order cannot be completed it moves to FAILED and the customer is told why. The release must itself be idempotent, because a recovery process may run it twice.

C4. The order lifecycle as a state machine

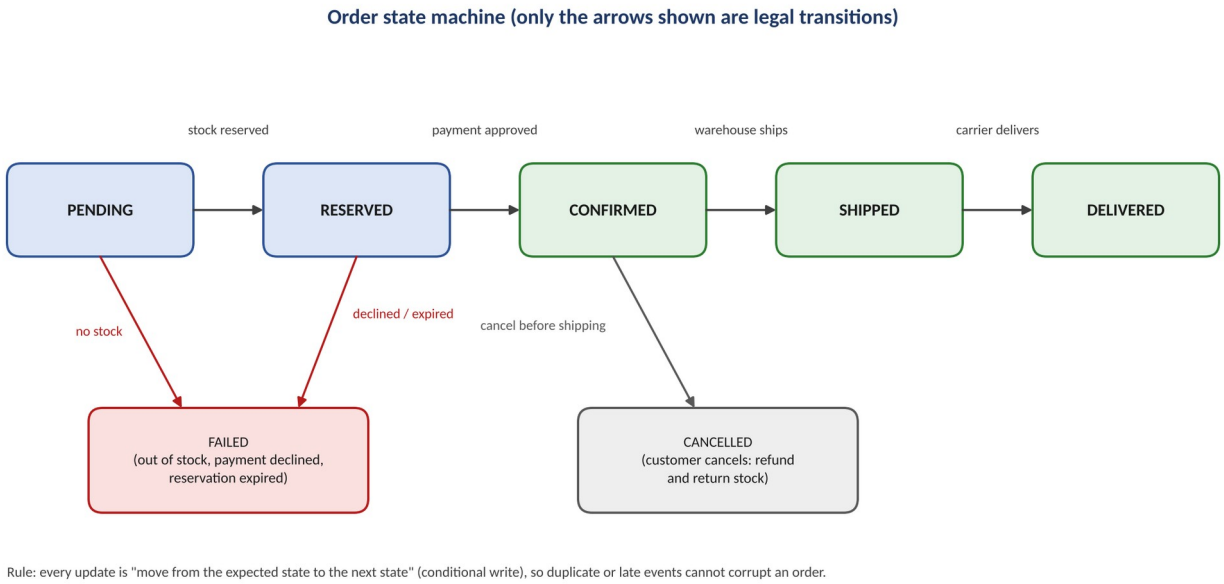


Figure 4. Order states and legal transitions

Modeling the order as a state machine gives you three things: a complete audit trail (every transition is recorded with a timestamp), protection against bad updates (a late event cannot move a FAILED order back to CONFIRMED), and a clean way to find stuck orders (for example "RESERVED for more than 20 minutes"). Implement each transition as a **conditional update**: "set status = CONFIRMED where status = RESERVED".

C5. The two-minute HLD talk track

> **Say it like this:** "I split the system by consistency need. The cart is a DynamoDB table keyed by user and favors availability; if two devices edit it I merge by union. Browse traffic is served from cache and CDN. Checkout goes through an Order service that runs a saga. It records the order as PENDING with the client's idempotency key, asks the Inventory service to reserve every line with an atomic conditional update and a 15 minute expiry, then authorizes payment through the gateway using the order id as the reference. On success it commits the reservation, marks the order CONFIRMED and writes an event to an outbox in the same transaction; a relay publishes it to an event bus for fulfillment, notifications and search. If payment fails, the Order service releases the reservation and marks the order FAILED. A reconciliation job finds stuck orders and repairs them. Inventory and orders are strongly consistent; everything downstream is eventually consistent. At 10x load I would scale stateless services, cache the browse path, and use sharded counters for hot SKUs."

Part D. Low-Level Design (LLD)

Interviewers often switch to classes after the HLD: "show me how Inventory and checkout would look in code". The design below is deliberately small and testable. It uses in-memory maps where production would use a database, but the concurrency rules are the same: the check and the update are one atomic step, reservations are all-or-nothing, and every operation is safe to repeat.

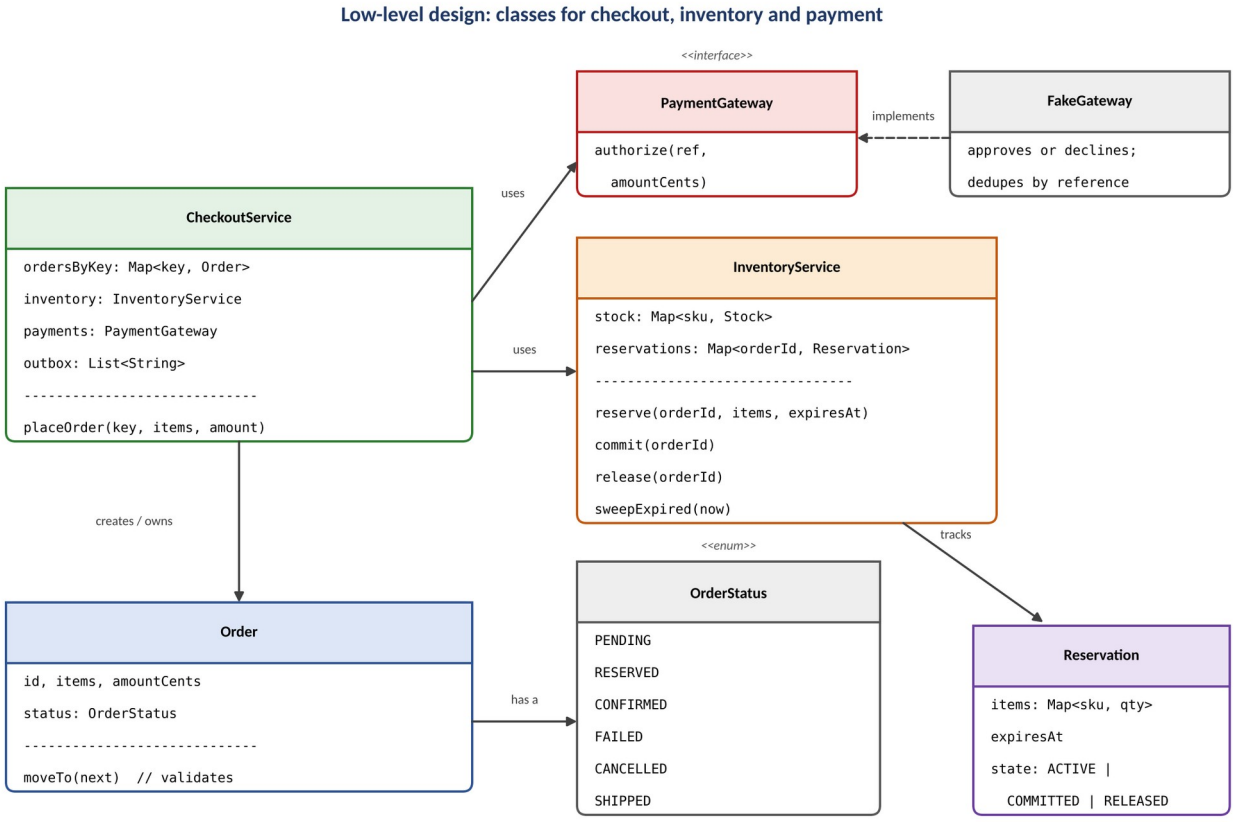


Figure 5. Class diagram for checkout, inventory and payment

Class	Responsibility	Key design choice
OrderStatus (enum)	The lifecycle and the legal transitions	`canMoveTo` makes illegal transitions impossible
Order	Holds id, items, amount and status	`moveTo` is synchronized and validates the transition
Reservation	One hold of stock for one order	Has a state (ACTIVE, COMMITTED, RELEASED) so commit and release run at most once

Class	Responsibility	Key design choice
InventoryService	Reserve, commit, release, expire	Per-SKU lock around check-and-update; all-or-nothing with rollback; idempotent
PaymentGateway (interface)	Authorize a payment	An interface so tests use a fake and production uses the real gateway
CheckoutService	Runs the saga for one order	Idempotency map first, then reserve, pay, commit or compensate

D1. OrderStatus and Order

```
enum OrderStatus {
    PENDING, RESERVED, CONFIRMED, FAILED, CANCELLED, SHIPPED;

    boolean canMoveTo(OrderStatus next) {
        switch (this) {
            case PENDING:   return next == RESERVED || next == FAILED;
            case RESERVED:  return next == CONFIRMED || next == FAILED;
            case CONFIRMED: return next == SHIPPED || next == CANCELLED;
            default:        return false;   // FAILED, CANCELLED, SHIPPED: final
        }
    }
}

class Order {
    final String id;
    final Map<String, Integer> items;
    final long amountCents;
    private OrderStatus status = OrderStatus.PENDING;

    Order(String id, Map<String, Integer> items, long amountCents) {
        this.id = id; this.items = items; this.amountCents = amountCents;
    }
    synchronized OrderStatus getStatus() { return status; }
    synchronized void moveTo(OrderStatus next) {
        if (!status.canMoveTo(next)) {
            throw new IllegalStateException(
                    status + " -> " + next + " is not allowed");
        }
        status = next;
    }
}
```

The `switch` is the transition table from Figure 4 in code. `moveTo` throws `IllegalStateException` for anything else, so a bug or a late duplicate event cannot silently corrupt an order.

D2. InventoryService: no overselling

```
class Reservation {
    enum State { ACTIVE, COMMITTED, RELEASED }
    final Map<String, Integer> items;
    final long expiresAtMs;
    State state = State.ACTIVE;
    Reservation(Map<String, Integer> items, long expiresAtMs) {
        this.items = items; this.expiresAtMs = expiresAtMs;
    }
}
```

```
class InventoryService {
    private static class Stock { int available; int reserved; }

    private final Map<String, Stock> stock = new ConcurrentHashMap<>();
    private final Map<String, Reservation> reservations = new ConcurrentHashMap<>();

    void addStock(String sku, int qty) {
        Stock s = stock.computeIfAbsent(sku, k -> new Stock());
        synchronized (s) { s.available += qty; }
    }
    int available(String sku) {
        Stock s = stock.get(sku);
        synchronized (s) { return s.available; }
    }
    int reserved(String sku) {
        Stock s = stock.get(sku);
        synchronized (s) { return s.reserved; }
    }

    // All-or-nothing. Calling it again for the same order is safe (idempotent).
    boolean reserve(String orderId, Map<String, Integer> items, long expiresAtMs) {
        if (reservations.containsKey(orderId)) return true;
        List<String> done = new ArrayList<>();
        for (Map.Entry<String, Integer> line : new TreeMap<>(items).entrySet()) {
            if (reserveLine(line.getKey(), line.getValue())) {
                done.add(line.getKey());
            } else {
                // roll back the lines already reserved
                for (String sku : done) unreserveLine(sku, items.get(sku));
                return false;
            }
        }
        reservations.put(orderId, new Reservation(items, expiresAtMs));
        return true;
    }

    // The in-memory equivalent of: UPDATE ... WHERE available >= qty
    private boolean reserveLine(String sku, int qty) {
        Stock s = stock.get(sku);
        if (s == null) return false;
        synchronized (s) {
            if (s.available < qty) return false;
            s.available -= qty;
            s.reserved += qty;
            return true;
        }
    }
    private void unreserveLine(String sku, int qty) {
        Stock s = stock.get(sku);
        synchronized (s) { s.available += qty; s.reserved -= qty; }
    }

    // payment succeeded: the stock is sold
    void commit(String orderId) {
        Reservation r = reservations.get(orderId);
        if (r == null) return;
        synchronized (r) {
            if (r.state != Reservation.State.ACTIVE) return;
            for (Map.Entry<String, Integer> line : r.items.entrySet()) {
```

```
                Stock s = stock.get(line.getKey());
                synchronized (s) { s.reserved -= line.getValue(); }
            }
            r.state = Reservation.State.COMMITTED;
        }
    }

    // compensation or expiry: give the stock back
    void release(String orderId) {
        Reservation r = reservations.get(orderId);
        if (r == null) return;
        synchronized (r) {
            if (r.state != Reservation.State.ACTIVE) return;
            for (Map.Entry<String, Integer> line : r.items.entrySet()) {
                unreserveLine(line.getKey(), line.getValue());
            }
            r.state = Reservation.State.RELEASED;
        }
    }

    int sweepExpired(long nowMs) {                    // run periodically
        int released = 0;
        for (Map.Entry<String, Reservation> e : reservations.entrySet()) {
            Reservation r = e.getValue();
            boolean expired;
            synchronized (r) {
                expired = r.state == Reservation.State.ACTIVE
                        && r.expiresAtMs <= nowMs;
            }
            if (expired) { release(e.getKey()); released++; }
        }
        return released;
    }
}
```

Reasoning to say aloud:

- **reserveLine is the atomic check-and-update.** It holds the SKU lock while it tests `available >= qty` and subtracts. It is the in-memory version of `UPDATE ... WHERE available >= :qty` or a DynamoDB `ConditionExpression`.
- **All or nothing.** A multi-line order reserves line by line; if line 3 fails, lines 1 and 2 are rolled back before returning false. Iterating in sorted SKU order gives a consistent order of locking across threads.
- **Idempotent reserve.** Calling `reserve` again for the same order id returns true without taking stock twice, so a retried saga step is safe.
- **Commit and release run at most once.** The reservation state is checked under the reservation lock, so a duplicate release (for example from the sweeper and the saga at the same time) cannot return the same units twice.
- **sweepExpired** is the background job that frees abandoned holds, so a customer who closes the tab does not lock stock forever.

D3. CheckoutService: idempotent saga

```
interface PaymentGateway {
    boolean authorize(String reference, long amountCents);   // true = approved
}

class CheckoutService {
    private final Map<String, Order> ordersByKey = new ConcurrentHashMap<>();
```

```
    private final InventoryService inventory;
    private final PaymentGateway payments;
    final List<String> outbox =
            Collections.synchronizedList(new ArrayList<String>());
    private final AtomicLong seq = new AtomicLong();
    private final long reservationTtlMs;

    CheckoutService(InventoryService inventory, PaymentGateway payments,
                    long reservationTtlMs) {
        this.inventory = inventory;
        this.payments = payments;
        this.reservationTtlMs = reservationTtlMs;
    }

    Order placeOrder(String idempotencyKey, Map<String, Integer> items,
                     long amountCents, long nowMs) {
        Order fresh = new Order("O-" + seq.incrementAndGet(), items, amountCents);
        Order existing = ordersByKey.putIfAbsent(idempotencyKey, fresh);
        if (existing != null) return existing;   // retry: original result

        if (!inventory.reserve(fresh.id, items, nowMs + reservationTtlMs)) {
            fresh.moveTo(OrderStatus.FAILED);        // out of stock
            return fresh;
        }
        fresh.moveTo(OrderStatus.RESERVED);

        boolean approved = payments.authorize(fresh.id, amountCents);
        // If authorize() throws or times out we do NOT release:
        // the outcome is unknown. The order stays RESERVED and a
        // reconciliation job asks the gateway what happened.
        if (approved) {
            inventory.commit(fresh.id);
            fresh.moveTo(OrderStatus.CONFIRMED);
            // in a real DB: same transaction as the order update
            outbox.add("OrderConfirmed:" + fresh.id);
        } else {
            inventory.release(fresh.id);             // compensation
            fresh.moveTo(OrderStatus.FAILED);
        }
        return fresh;
    }
}
```

Reasoning to say aloud:

- **putIfAbsent on the idempotency key** is the guard. Exactly one thread (or one request) wins and runs the saga; every retry gets the same Order back and nothing is charged twice.
- **Failure paths are explicit.** Out of stock leads to FAILED with nothing to undo. A declined payment calls `release` (the compensation) and then FAILED.
- **Unknown payment outcome is not a failure.** If `authorize` throws or times out, the code deliberately does **not** release the stock: the customer may have been charged. The order stays RESERVED and a reconciliation job asks the gateway by reference. (Section E5 explains this.)
- **The outbox.** The list stands in for an outbox table; in a real database the order update and the outbox insert are one transaction.
- **A duplicate that arrives while the first request is still running** gets the same Order object back, possibly still PENDING; in production it would poll or wait for the final state.

D4. What was tested

This code was compiled and run (Java 8 compatible) with the scenarios below, repeated several times to catch timing problems. All passed.

Scenario	Expected	Result
400 concurrent buyers, 1 unit each, stock 100	Exactly 100 CONFIRMED, 300 FAILED, stock ends 0 available / 0 reserved, 100 charges, 100 outbox events	Passed
Order for A x2 and B x1 where B is out of stock	FAILED, and A is back to its original 5 (rollback)	Passed
Same idempotency key sent twice	Same order returned, one charge, one unit taken	Passed
50 threads send the same key at once	One order id, one charge	Passed
Payment declined	Reservation released, order FAILED, stock restored	Passed
Reservation expiry sweep	Nothing released before expiry; stock restored at expiry; later release or commit is a no-op	Passed
PENDING to CONFIRMED directly	Rejected with an exception	Passed

Part E. Deep dives with diagrams

E1. Preventing overselling

The bug is a **read-then-write race**: two requests read the same stock value and both decide there is enough. The cure is to make "check" and "update" a single atomic operation that the database serializes.

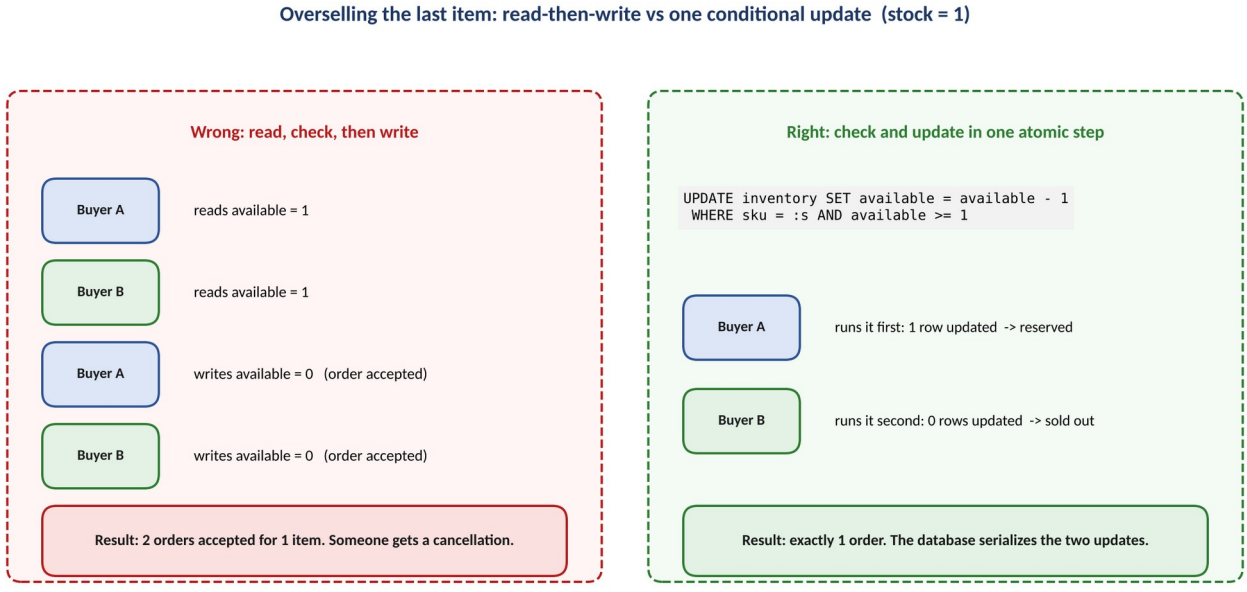


Figure 6. The last unit: race condition versus conditional update

```
-- SQL
UPDATE inventory
   SET available = available - :qty,
       reserved  = reserved  + :qty
 WHERE sku = :sku AND fc_id = :fc AND available >= :qty;
-- rows affected = 0  ->  not enough stock, fail this line

// DynamoDB equivalent
UpdateItem  Key: {sku, fc}
  UpdateExpression:    SET available = available - :q, reserved = reserved + :q
  ConditionExpression: available >= :q
// ConditionalCheckFailedException  ->  not enough stock
```

Alternatives you can mention: optimistic locking with a version column (retry on conflict), or `SELECT ... FOR UPDATE` in SQL. The conditional update is simplest and needs no explicit lock held across network calls.

E2. Hot items in a flash sale

Even with a conditional update, 5,000 writes per second on one row exceed what a single partition can serve, so requests queue or are throttled. The fix is to spread one counter across several partitions.

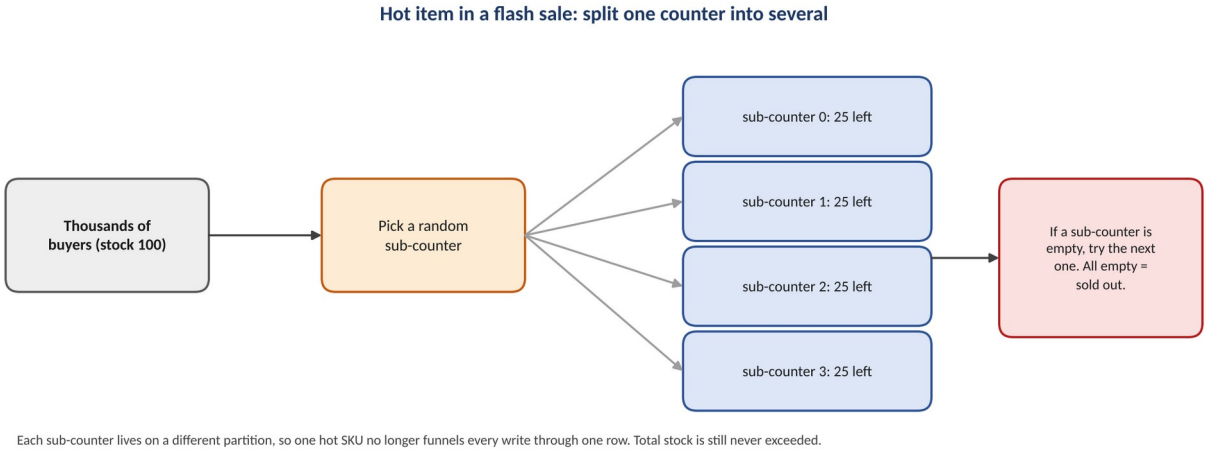


Figure 7. Splitting a hot SKU into sub-counters

Option	How it works	Trade-off
Sharded counters	Split stock 100 into 4 rows of 25. Each buyer picks one at random and falls back to the others when empty	Spreads load. A buyer may see sold out slightly early on a busy shard if the fallback is skipped; total stock is never exceeded.
Queue per hot SKU	Route purchase attempts to a queue that one worker drains in order	Very safe and fair, but adds latency and a throughput ceiling per SKU
Purchase tokens at the edge	Pre-issue exactly N tokens (N = stock) and reject the rest before they reach the database	Protects the backend; needs care for tokens that are never used (expiry returns them)

Also say: the "only 3 left" label on the product page can be approximate and cached. Only the reservation must be exact.

E3. Idempotency: retries must be harmless

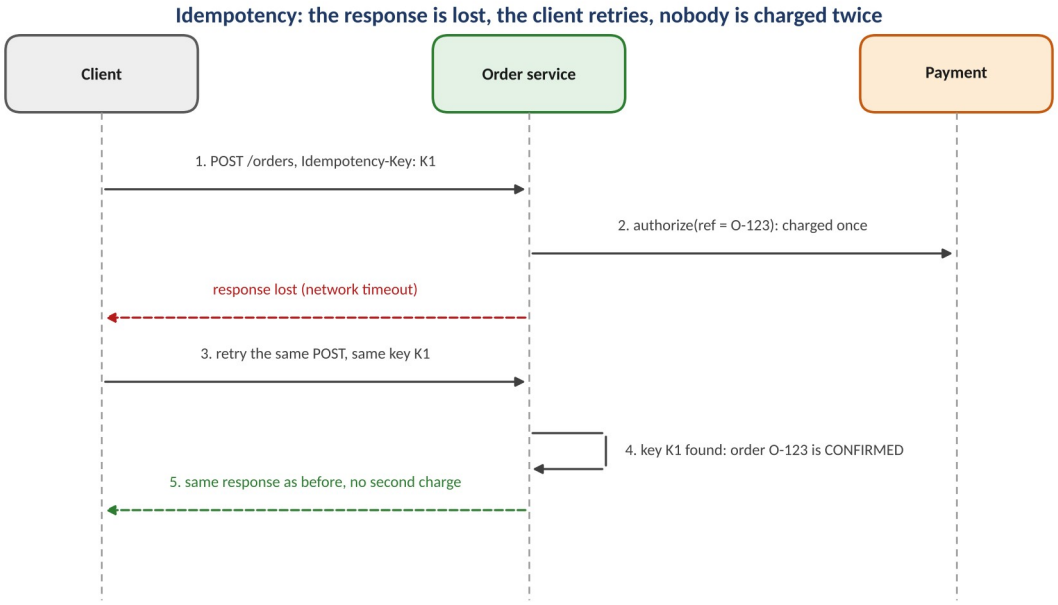


Figure 8. A lost response and a retry

- **Where the key is stored:** in the orders table with a unique constraint on the key, written in the same step that creates the order. A second insert with the same key fails and the existing order is returned.
- **What is stored:** the key, the order id and the final response, so the retry returns exactly the same answer.
- **Key lifetime:** keep keys at least as long as clients might retry (for example 24 hours).
- **Defense in depth:** pass the order id to the payment gateway as its own idempotency reference, so even a bug in your layer cannot double charge.
- **Consumers too:** events can be delivered twice, so consumers (fulfillment, notifications) check "have I already handled this order id?" before acting.

E4. Reliable events with the transactional outbox

Naive code saves the order, then publishes an event. If the service crashes between the two, the order exists but fulfillment never hears about it. Publishing first is also wrong: the event could announce an order that was never saved. The two systems (database and bus) cannot share a transaction.

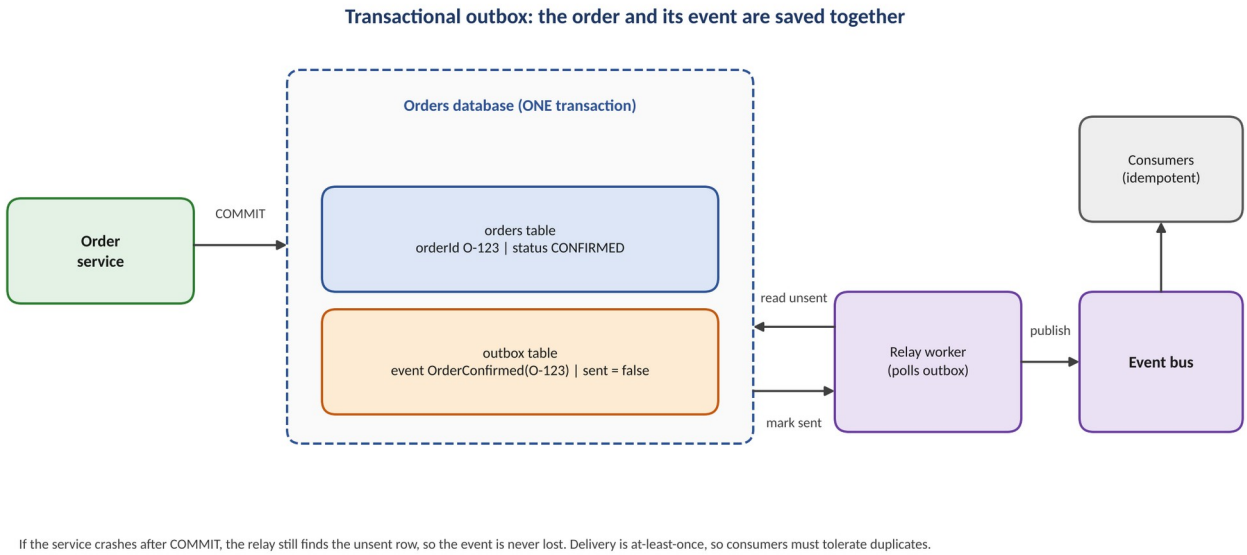


Figure 9. Transactional outbox

1. In one local database transaction, update the order and insert an event row into the outbox table. Both commit or neither does.
2. A relay process reads unsent outbox rows, publishes them to the bus and marks them sent.
3. If the relay crashes after publishing but before marking sent, the event is published again. That is **at-least-once** delivery, which is why consumers must be idempotent.

E5. Payment timeout: the unknown outcome

The hardest failure is not "declined", it is "I do not know". The gateway call times out; the customer may or may not have been charged. Treat three outcomes differently:

Gateway result	Meaning	Action
Approved	Money is authorized	Commit the reservation, confirm the order
Declined	Definitely no charge	Release the reservation, fail the order (compensation)
Timeout or error	**Unknown**	Do not release and do not blindly retry the charge. Keep the order RESERVED, extend the reservation, and let a reconciler query the gateway by reference, then continue as approved or declined.

A **reconciliation job** is the safety net for the whole saga. It scans for orders stuck in an intermediate state for too long (RESERVED with no payment answer, CONFIRMED with no fulfillment event, reservations without an order) and repairs them. Mention it explicitly; it separates production-minded answers from textbook ones.

E6. Reservation expiry

- **Why reservations exist:** stock must be held while the customer pays, but reserving at add-to-cart would let browsers lock all the stock. Reserve at checkout, hold for a short time.
- **Expiry:** each reservation has `expiresAt`. A sweeper (or a delayed queue message per reservation) releases expired holds. The release is idempotent, so running it twice is safe.
- **Race between expiry and payment success:** payment approves at the same moment the hold expires. Commit first checks the reservation is still ACTIVE; if it was released, re-reserve (if stock remains) or refund and fail the order.

E7. Cart, pricing and allocation

- **Cart:** one DynamoDB item per user (or per guest cookie id) holding a map of SKU to quantity. Concurrent edits from two devices are merged (for example take the larger quantity per SKU). Guest carts get a TTL (for example 30 days) and are merged into the user cart at login.
- **Price at checkout:** the catalog is eventually consistent, so re-read the price when the order is created. If it changed, ask the customer to confirm instead of silently charging a different amount.
- **Which warehouse:** an allocation step chooses the FC (nearest with stock, or split the shipment). Keep it behind the Inventory interface so checkout does not need to know.
- **Authorize then capture:** authorize at checkout, capture when the order ships. If the order is cancelled before shipment, void the authorization.

Part F. One complete example from start to finish

Priya orders 2 phone cases (SKU-42) and 1 charger (SKU-77). Use a story like this to rehearse; concrete numbers make the explanation stick.

Step	What happens	State afterwards
1	Priya clicks Place order. Client sends POST /orders with Idempotency-Key K1.	Order O-901 PENDING; key K1 stored
2	Inventory reserves SKU-42 x2 (available 10 -> 8, reserved 0 -> 2) and SKU-77 x1 (available 1 -> 0, reserved 0 -> 1). Hold expires in 15 minutes.	Order RESERVED
3	Meanwhile Dev tries to buy the last SKU-77. His conditional update finds available = 0 and fails: "out of stock". No oversell.	Dev's order FAILED, nothing to undo
4	Payment service calls the gateway with reference O-901. The response is slow and the client times out.	Priya's app shows a spinner
5	Priya's app retries with the same key K1. Order service finds K1, returns O-901 in its current state; no second reservation, no second charge.	Still RESERVED
6	The gateway approves. Order service commits the reservation (reserved 2 -> 0 and 1 -> 0, those units are sold) and marks the order CONFIRMED, writing an OrderConfirmed outbox row in the same transaction.	Order CONFIRMED
7	The relay publishes OrderConfirmed. Fulfillment picks the items; the notification service emails Priya. Both check "already handled O-901?" first.	Events delivered at least once
8	The warehouse ships; the order moves to SHIPPED, and the authorization is captured.	Order SHIPPED
9	Had the gateway declined instead at step 6: release the holds (available 8 -> 10 and 0 -> 1), mark FAILED, notify Priya. Had the app crashed at step 6: the reconciler finds O-901 RESERVED for 20 minutes, asks the gateway by reference, and completes either path.	Safe in every case

Part G. Connecting this to a supply chain team

Your next interview is for a team in AWS Infrastructure Supply Chain. I do not know that team's exact systems, so treat this as an illustration of how to transfer the ideas, not as a description of their architecture. Supply chain software moves physical goods (servers, networking gear, components) through purchasing, transit, receiving, storage and installation, and its software problems resemble the ones above.

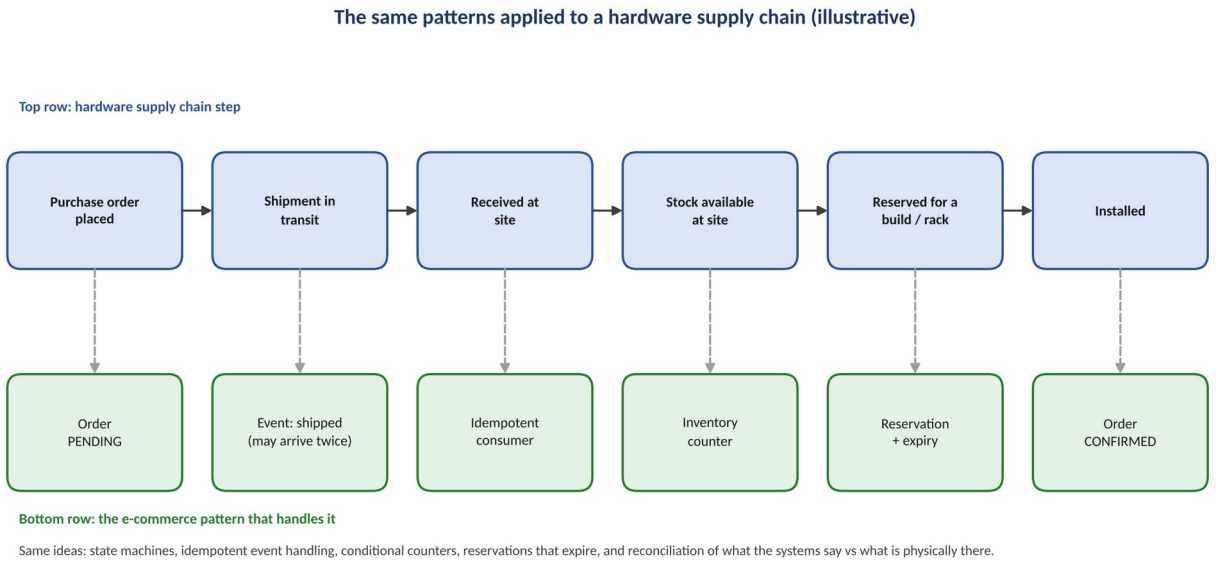


Figure 10. The same patterns mapped onto a hardware supply chain

E-commerce idea	Supply chain counterpart (illustrative)
Order state machine	Purchase order or shipment lifecycle: requested, approved, shipped, received, installed. Illegal transitions are rejected; late events are handled safely.
Inventory counter with conditional update	Stock of a component at a site. Several teams or automated jobs claim the same pool, so claims must be atomic.
Reservation with expiry	Hardware set aside for a planned build or rack. If the plan slips, the hold expires and the stock returns to the pool.
Idempotent event handling	Carrier or warehouse systems send duplicate or out-of-order status events. Applying the same event twice must not change inventory twice.
Outbox and at-least-once delivery	Reliable propagation of "received" or "allocated" events to planning and reporting systems.
Reconciliation job	Systems of record disagree with what is physically on the shelf. A scheduled comparison finds and fixes the differences (cycle counts, audits).
Skew and hot keys	A scarce, in-demand component or a large launch creating contention on one record.

> **Say it like this:** "This is the same shape of problem as order and inventory: a lifecycle with strict state transitions, shared counters that several actors claim atomically, reservations that expire, duplicate or out-of-order events, and a reconciliation loop between the software and the physical world. In my own projects I applied this pattern when ..." (finish with a real example from your experience).

> **Use real stories.** The strongest move in the interview is to describe a time you handled duplicate messages, race conditions on shared state, or data that disagreed between systems in your own microservices work. Prepare one such story with numbers (volume, failures avoided, time saved).

Part H. Reliability, trade-offs and rehearsal

H1. What happens when something fails

Failure	Effect	Mitigation
Order service crashes mid-saga	Order left in PENDING or RESERVED	Every step is persisted as a state; a recovery worker resumes or compensates; reservations expire anyway
Payment gateway timeout	Unknown whether the customer was charged	Keep RESERVED, extend the hold, query the gateway by reference before releasing or retrying
Inventory database slow or throttled	Checkout latency rises	Timeouts, retries with backoff and jitter, shed load, queue hot SKU traffic
Event bus unavailable	Fulfillment and email delayed	Outbox keeps events safe in the database; the relay catches up when the bus returns
Duplicate event delivery	Double shipment or double email	Idempotent consumers keyed by order id and event id
Cache stale on product page	Customer sees an old price or stock count	Re-validate price and reserve stock at checkout, which are authoritative

Failure	Effect	Mitigation
Region outage	Service unavailable in that region	Multi-AZ by default; a second region for orders with global tables or per-region partitioning of inventory

H2. If traffic grows 10x (Prime Day)

- **Browse path:** cache and CDN; scale stateless services horizontally; pre-warm caches for promoted items.
- **Checkout:** isolate it from browse traffic with separate capacity and its own rate limits, so a surge of browsing cannot starve payment.
- **Inventory:** partition by SKU and FC; use sharded counters for hot SKUs; load-test the checkout path before the event.
- **Load shedding:** switch off non-essential features (recommendations, personalization) before touching checkout.
- **Queues:** put non-critical work (emails, analytics) behind queues so spikes are absorbed.

H3. Trade-offs to state

Decision	Chosen	Alternative and its cost
Cart consistency	Available, merge on conflict	Strong consistency would fail the cart during partitions, hurting sales for no real gain
Inventory consistency	Strong, conditional updates	Eventual consistency oversells under load
Cross-service transaction	Saga with compensation	Two-phase commit is slow, fragile and blocks on failures
When to take stock	Reserve at checkout, commit at payment	Decrement only after payment oversells; reserve at add-to-cart locks stock for browsers
Oversell vs undersell	Prefer a small undersell (safety buffer)	Overselling forces cancellations and damages trust
Event delivery	Outbox with at-least-once delivery and idempotent consumers	Exactly-once delivery is not practical across systems
Orchestration style	Orchestrated saga (one service drives it)	Choreography (services react to each other's events) has fewer central points but is harder to follow and debug

H4. Follow-up questions with model answers

Interviewer asks	Strong answer
How do you guarantee no overselling?	One atomic conditional update per line (`available >= qty`), all-or-nothing across lines with rollback, and a reservation with expiry. Strong consistency on the inventory store.
How do you avoid charging twice?	Client idempotency key stored with the order under a unique constraint, plus our order id passed to the gateway as its own idempotency reference.

Interviewer asks	Strong answer
What if the service crashes after payment but before saving the order?	Create the order as PENDING first, so there is always a record. On restart, a recovery job finds RESERVED orders, queries the gateway by reference and completes or compensates.
Why a saga and not a distributed transaction?	Two-phase commit locks resources across services and fails badly when one participant is down. A saga keeps each service local and handles failure with compensations.
How do you handle a flash sale on one item?	Shard the stock counter across partitions or queue purchase attempts for that SKU; show approximate stock on the page, exact stock at reservation.
How do you handle cancellation?	Another transition (CONFIRMED to CANCELLED) with compensations: void or refund the payment and return stock to available. It is idempotent like everything else.
How do you keep the search index in sync?	Publish product change events through an outbox to a stream; the indexer consumes them. Eventual consistency is fine.
How would you monitor it?	Order success rate, checkout latency p99, reservation conflict rate, orders stuck by state and age, payment error rate, outbox lag, queue depth, reconciliation repairs per hour.

H5. Common mistakes

- **Read then write on inventory** without naming the race and the atomic fix.
- **One consistency level for everything.** Say which parts are strong and which are eventual, and why.
- **No idempotency story.** Retries are the normal case, not the exception.
- **Treating a payment timeout as a failure** and releasing stock while the customer was actually charged.
- **Saving the order and publishing the event separately** with no outbox.
- **Forgetting reservation expiry,** so abandoned checkouts lock stock forever.
- **Skipping reconciliation.** Sagas need a repair loop.

H6. One-page cheat sheet

Cheat sheet	Answer
Core promises	No oversell, no double charge, no lost order
Numbers	115 orders/s average, 1,200 at peak; about 7 TB of orders per year; hot SKU about 5,000 writes/s on one key
Consistency map	Cart: available. Catalog: eventual. Inventory, orders, payment: strong. Events: eventual
Checkout	Saga: PENDING, reserve (conditional update, expiry), authorize, commit, outbox event; compensate on failure
Idempotency	Client key stored with the order; our reference to the gateway; idempotent consumers
Hot SKU	Sharded counters, queue per SKU or edge tokens
Failure rule	Declined: release. Unknown: keep reserved and reconcile
Operations	Stuck-order alarms, reconciliation job, outbox lag, checkout p99

Next Steps

How to practice these two chapters

1. Read the chapter once for understanding. Do not memorize numbers; understand why each number exists, and run the Java LLD yourself.
2. Close the document and redraw the HLD from memory. Check it against the figure.
3. Say the two-minute HLD talk track aloud, timing yourself.
4. Pick three deep dives (idempotency, oversell race with the conditional update, payment timeout and reconciliation) and explain each in two minutes.
5. Ask a friend to play interviewer using the follow-up tables, or answer them out loud and compare.

Questions 4 and 5 (news feed and distributed cache) remain in the Top 5 guide and can be expanded in the same format next.